

CyberGuard : “ Real-Time Zero-Day Threat Detection for GitHub with Chrome Extension ”

Rashi Jain

Neha Mahajan

Suridhi Gupta

Dr D. Y. Patil School of Science &
Technology,Pune, India
jainrashil52@gmail.com

Dr D. Y. Patil School of Science &
Technology,Pune, India
nehaashokmahajan27@gmail.com

Dr D. Y. Patil School of Science &
Technology,Pune, India
suridhigupta2022@gmail.com

***Abstract—** Software weaknesses are a very big issue in today's application development, and they are mostly causing problems like data leaks, system crashes and security dangers. The old ways that are used for finding these weaknesses are mostly reactive, which means they don't give you real time and proactive information about the threats. So to fix this problem, what we did is we introduced CyberGuard, which is basically a system for predicting vulnerability risk in real time. And the way it works is that it is based on a tri-hybrid approach that combines multiple things together. So the system uses machine learning, CVSS scores for severity, and code scanning based on rules, and all these things together they make the predictions more accurate and trustworthy. So the text descriptions and code are first processed using preprocessing techniques and TF-IDF to extract the features, and then these features are classified using a XGBoost model. At the same time a code scanner it also runs to find coding patterns that are not safe. Then all these parts they are put together by a fusion engine which creates a final risk level and also helps in making better decisions. So this system it also allows you to use many different ways to give input, like you can directly put code or description, or you can scan GitHub repositories, or you can also upload files or ZIP folders for a complete analysis. And on top of all this, a Gemini API-based generative AI module provides easy-to-understand explanations, remediation guidance, and automatic suggestions on how to fix the problems. The system also generates reports that you can download, and it also keeps the old data for future reference. And we also put it as a Chrome extension for real time GitHub integration, which helps in finding vulnerabilities early and makes the software more safe and reliable overall.*

***Keywords:** zero-day vulnerability detection, real-time risk prediction, GitHub security scanning, Chrome extension integration, XGBoost classification, rule-based code analysis, explainable AI, Gemini API integration, cybersecurity SaaS*

I. INTRODUCTION

Zero-day attacks basically exploit vulnerabilities that are unknown to the public and the cybersecurity community at the time of the attack, and because of this the threat actors are able to stay undetected by the normal defense systems. And since the attack signatures and exploit patterns are not available beforehand, the traditional signature based and standalone rule-based detection approaches are mostly not effective in identifying these kinds of threats[1]. So with the

increasing dependence on web applications, cloud platforms and distributed software systems, software vulnerabilities have become one of the biggest causes of data breaches, financial loss and service disruption. And especially things like insecure coding practices, unsafe input handling and dynamic execution patterns are increasing the risk of exploitation a lot[2].

Recent research shows that machine learning based detection methods are capable of capturing the statistical and behavioral characteristics of attacks, which makes them very suitable for identifying zero-day threats and unseen malicious patterns. And many studies have also shown that models like Random Forest and XGBoost perform really well in classifying suspicious and previously unseen attack behaviors [3][4].

To fix these limitations, what we did is we proposed CyberGuard, which is basically a real time vulnerability risk prediction framework that combines machine learning based severity classification, rule based code scanning and an intelligent recommendation layer for remediation and zero-day threat insights together. So the system supports multiple analysis modes like direct code input, GitHub repository scanning and file based analysis, and this helps in enabling early and practical vulnerability assessment within real development workflows.

The major contributions of this work are as follows. First thing is that we made a machine learning driven risk prediction engine for classifying vulnerability severity levels. Second thing is that we built a hybrid code scanning and zero-day pattern insight framework. Third thing is that we also added an intelligent recommendation layer for giving fix suggestions and threat explanations. And the fourth thing is that we made a real time downloadable security report generation mechanism that helps the users to get a complete report of the vulnerabilities found in their code.

II. RELATED WORK

In this section we are reviewing the major approaches that have been proposed in existing literature for vulnerability detection and zero-day threat analysis.

We have organized this whole discussion into some key areas which we will go through one by one. So the first area is rule based and signature based detection systems. The second area is statistical and behavioral analysis approaches. The third area is machine learning based detection techniques. The fourth area is NLP and generative AI based analysis methods. And the fifth area is the research gap and motivation for our proposed work. So basically this whole review helps us to clearly establish the limitations that are there in the existing systems, and it also highlights why there is a need for a hybrid framework like CyberGuard. So first we talk about rule based and signature based detection systems.

These systems basically work by matching known patterns and signatures of vulnerabilities. So if a known attack pattern is found in the code or the network traffic, the system flags it as a threat. But the big problem with these systems is that they can only detect threats that they already know about. So for zero-day attacks where the pattern is completely new and unknown, these systems mostly fail to detect anything.

A. Rule-Based and Signature-Based Detection Systems

The rule based and signature based detection systems are basically the oldest and most widely used approaches in cybersecurity defense. So the way these systems work is that they match the observed behaviors, code patterns or network signatures against a predefined database of known vulnerabilities and exploit signatures. Basically if something in the code or network matches with something that is already in the database, the system flags it as a threat.

These approaches are used a lot in things like intrusion detection systems, malware scanners and static code analysis tools, and the main reason for this is that they are fast, easy to understand and they don't need very high computational resources. So signature based methods are especially very good and effective at detecting repeated attack patterns, known CVEs and exploit payloads that are already documented in the security databases.

But the thing is these systems have one very big limitation, which is that they can only detect threats that they already know about. So if a completely new and unknown vulnerability comes up, like a zero-day attack, these systems mostly fail to detect it because there is no matching signature available in their database. So this highlights the

need for a more advanced hybrid approach capable of handling both known and previously unseen threats more effectively[5].

B. Statistical and Behavioral Analysis Approaches

To fix the limitations of static signatures, what the researchers did is they introduced statistical and behavioral analysis methods. So these methods basically focus on finding deviations from the normal behavior of the system or the software. So instead of matching known signatures, these techniques look at things like execution flow, access frequency, anomaly thresholds and behavioral distributions to detect any suspicious activity. And these approaches are widely used in anomaly based intrusion detection systems and insider threat monitoring frameworks.

The good thing about behavioral systems is that they are capable of identifying previously unseen threats, because what they do is they first learn the normal patterns and then they flag anything that is showing a significant deviation from that normal pattern. So for example things like unusual API usage, dynamic code execution or abnormal repository modifications can all indicate potential vulnerabilities, and these systems are able to catch these kinds of things. But even with this advantage, these methods still have some problems. So one big problem is that they often have very high false positive rates, which means they are flagging a lot of normal activities as suspicious.

And another problem is that they are not very robust across different and heterogeneous environments, because the normal behavior can vary a lot from one system to another. And also it is very difficult to define anomaly thresholds that are reliable and work well in all situations. So because of all these limitations, a more advanced hybrid approach is required[6].

C. Machine Learning-Based Detection Techniques

Some recent research in cybersecurity strongly shows that machine learning is very useful for vulnerability prediction and zero-day attack detection. So supervised learning algorithms like Random Forest, XGBoost, Naive Bayes and Logistic Regression, have all shown really good and promising performance in classifying suspicious code patterns and predicting severity levels.

Therefore, machine learning approaches are especially very suitable for zero-day threat prediction, and the reason for this is that they learn statistical and semantic patterns from historical datasets, which helps them to detect previously unseen exploit behaviors that normal rule based systems would completely miss. And many studies have also shown that ensemble methods like Random Forest give very high

accuracy, better generalization and good robustness for attack classification tasks[7]. But even with all these good things, the current machine learning systems still have some challenges and limitations.

So one big problem is dataset imbalance, which means that in most security datasets the number of normal samples is much more than the malicious ones, and this affects the performance of the model. Another problem is model explainability, because most machine learning models are like a black box and it is very difficult to understand why they made a certain decision. And then there is also the problem of limited transferability across different software domains, which means a model trained on one type of software may not work well on another type. And lastly these models also depend a lot on high quality labeled datasets, which are not always easy to get [8].

D. NLP-Based Communication and Behavioral Analysis

Recently NLP, which is Natural Language Processing and generative AI models and API-based language systems ,have introduced a new direction in vulnerability analysis. These approaches use textual vulnerability descriptions, CVE summaries, source code comments and semantic code structures to extract contextual security intelligence.

NLP based methods are specially useful for processing vulnerability descriptions and mapping them to severity classes or exploit categories. And more recently API-driven generative AI systems have shown good promise in generating human readable explanations, fix recommendations, zero-day reasoning insights and secure code suggestions, and this improves both interpretability and usability for developers.

Hence, this layer is very relevant for modern software security products because it fills the gap between raw model predictions and actionable remediation guidance[9].

E. Research Gap and Motivation

Even though a lot of good progress has been made in all the areas we talked about above, the thing is most of the existing approaches are working in isolation. So what this means is that they are only focusing on one thing at a time, like either known signature matching, or anomaly detection, or standalone predictive models. And very little work has been done on integrated real time frameworks that combine all things together like severity prediction, rule based code scanning, zero-day threat insight generation and developer oriented remediation support all in a single deployable Chrome extension–based system.

This research gap is exactly what motivated us to propose the CyberGuard framework. So what CyberGuard does is it combines machine learning driven vulnerability severity

prediction with rule based code analysis and intelligent recommendation support together, and this helps in enabling proactive zero-day risk assessment in practical development workflows.

Table 1.1. Comparative Literature Survey

Approach	Strength	Limitation
Signature-Based	Fast known threat detection	Cannot detect zero-day
Statistical	Detects anomalies	High false positives
ML-Based	Predictive	Needs labeled data
NLP /Generative AI	Explainability	Depends on context
CyberGuard	Hybrid + zero-day insight	Proposed

So from this whole comparative analysis it is very clear that the existing systems are missing a proper integrated and proactive framework for real time zero-day vulnerability assessment, and from this comparative analysis, it is clear that existing systems lack an integrated and proactive framework for real-time zero-day vulnerability assessment.

III. METHODOLOGY

In this section we are describing the design and workflow of our proposed CyberGuard framework for real time zero-day vulnerability risk prediction.

The methodology is structured as a modular pipeline that consists of several steps, which are data acquisition, preprocessing, machine learning based severity prediction, rule based code scanning, intelligent recommendation support and report generation. So basically all these steps work together one after another to give the final result [10] .

The proposed architecture is designed to support multiple real world input modes, like direct code snippets, vulnerability descriptions, GitHub repository URLs and uploaded files or ZIP archives. So this means the user can give input in many different ways depending on what they need. And this modular design also improves the scalability, maintainability and practical deployment of the system in real software development workflows. And existing research also strongly supports the use of layered and hybrid architectures for proactive zero-day vulnerability assessment

A. System Architecture

The CyberGuard system that we proposed follows a tri-hybrid modular architecture that combines three main things together, which is machine learning, rule based scanning and intelligent recommendation support. So the overall workflow of the system is divided into the following layers that work together one by one.

The first one is the Input Layer, which is basically where the user gives the input to the system. Then the second one is the Preprocessing Layer, where the raw input data gets cleaned and prepared for further processing. Then the third one is the ML Prediction Layer, where the machine learning model analyzes the preprocessed data and predicts the severity level of the vulnerability. Then the fourth one is the Rule Based Detection Layer, where the system scans the code using predefined rules to find any unsafe coding patterns. Then the fifth one is the Gemini API-based intelligent recommendation layer, where the system gives easy to understand explanations and suggestions on how to fix the vulnerabilities that were found.

Therefore, all these layers work together in a smooth and organized way to make sure that the system is giving accurate, reliable and practical results for real time zero-day vulnerability assessment.

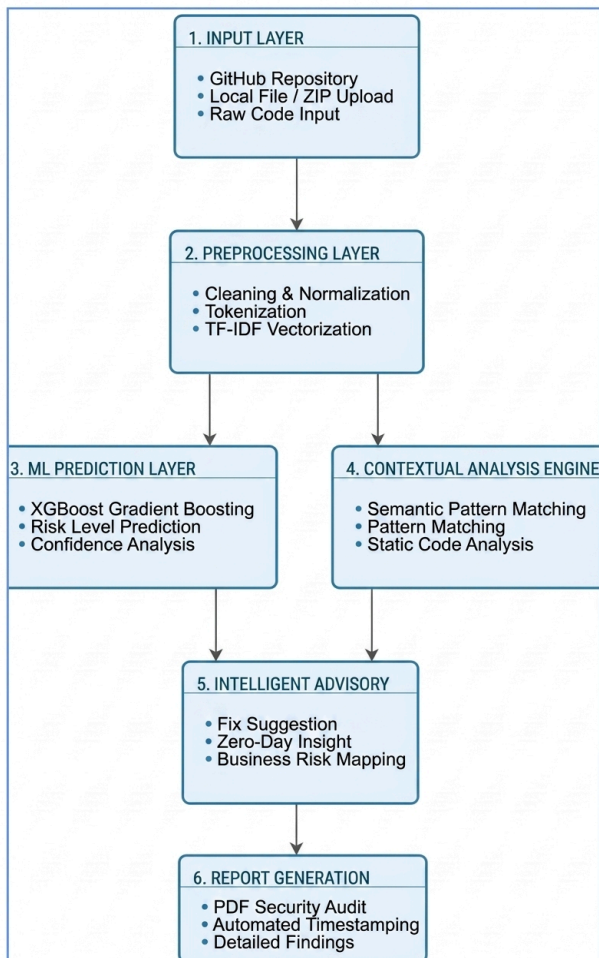


Fig 1. System Architecture

A.1. Input Layer

This layer accepts multiple forms of security analysis input, like source code snippets and textual vulnerability descriptions, GitHub repository links, and uploaded source files or ZIP files. So this multi-input design makes sure that the system can be integrated into both developer side code review workflows and repository level security assessment pipelines. So basically the user can give input in whatever way is most convenient for them.

A.2. Preprocessing Layer

The preprocessing module does several things like text cleaning, token normalization and special character removal, lowercase conversion and TF-IDF vector transformation. And for code files specifically, the system extracts suspicious API calls, execution patterns, unsafe input functions and shell execution commands. So preprocessing it is very critical and important for improving feature consistency and model prediction performance, and the same approaches are also widely used in vulnerability prediction literature[11][12].

A.3. ML Prediction Layer

This phase handles the actual risk assessment by feeding the cleaned data into our trained XGBoost classifier, and this classifier sorts the vulnerabilities into four main categories which are Low, Medium, High and Critical. So we chose XGBoost over other options because it provides the best balance of precision and recall, which is very important for security work. And by using weighted loss functions, we have also made the system better at handling False Negatives which are also called Type II errors, so that it can detect zero-day threats before they cause any serious problems.

A.4. Rule-Based Detection Layer

This detection layer has been designed to systematically find a range of common security vulnerabilities by looking for specific high risk function calls and coding patterns. So the key areas that it focuses on include things like the use of dynamic execution functions like `eval()` and `exec()`, unauthorized shell commands via `os.system()` or `subprocess` calls with `shell = True`, and common exploit vectors like SQL injection and unsafe input handling. So this layer helps to detect explicit vulnerabilities that may not always need the machine learning model to identify them.

A.5. Intelligent Recommendation Layer

This final output layer generates the final risk level, a findings summary, a Gemini API-generated remediation and

zero-day insight summary, and also a downloadable PDF audit report. So all these things together they support SaaS level usability and make the system ready for real world deployment. This layer is very important because it makes the system not just good at finding vulnerabilities but also very useful and practical for the developers who need to fix them.

A.6. Report Generation Layer

The final output layer generates the final risk level and findings summary, and along with this it also produces a Gemini API-generated remediation and zero-day insight summary. And on top of all this it also gives downloadable PDF audit reports which are very helpful for the developers to understand and fix the vulnerabilities.

So as we can see in **Fig 1**, the CyberGuard framework that we proposed follows a layered architecture for real time vulnerability risk assessment. So the system starts with multiple input sources, and then after that it goes through preprocessing and feature extraction. Then the processed data is analyzed through both the machine learning based prediction module and the rule based detection module at the same time. And then the outputs from both of these modules are combined together to generate a final risk score. And then this final risk score is further improved by adding intelligent recommendations and zero-day threat insights before the system produces a complete and comprehensive security audit report.

So overall this architecture makes sure that the framework is scalable, modular and ready for real world deployment. And It is a practical and useful solution for proactive vulnerability detection and developer oriented remediation support.

B. Data Collection

The dataset that we used in our proposed CyberGuard framework, we collected from the National Vulnerability Database which is also called NVD, and this is maintained by the National Institute of Standards and Technology which is NIST. So we retrieved this data using the official NVD CVE API v2.0, and this API provides structured JSON records of software vulnerabilities along with their severity and metadata information [15].

The API endpoint used for data collection was the official NVD CVE API v2.0 endpoint (/rest/json/cves/2.0). The data collection process was implemented in the data_loader.py module, where the CVE records they were fetched in batches using API pagination. So for this study what we did is we collected approximately 2,000 CVE records, and we collected this much data to support multi-class vulnerability

severity prediction and to make sure there is good representation for all the classification tasks like Low, Medium, High and Critical risk prediction [16].

Table 2.1 Important Dataset Features

Features	Description
cve_id	Unique vulnerability identifier
description	Textual vulnerability description
cvss_score	Severity score provided by NVD
risk_level	Low / Medium / High / Critical
published_date	CVE release date
cwe	Associated weakness type
references	External vulnerability links

Among all these attributes, the description and CVSS score are the most important features in our machine learning pipeline, while the risk level is used as the target class for vulnerability severity prediction.

```
Dataset shape: (2000, 4)

Columns: ['cve_id', 'description', 'cvss_score', 'risk_level']

Data types:
cve_id      object
description object
cvss_score  float64
risk_level  object
dtype: object

Null counts:
cve_id      0
description 0
cvss_score  34
risk_level  0
dtype: int64
```

Fig 2. Dataset Shape

	cve_id	description	cvss_score	risk_level
0	CVE-1999-0095	The debug command in Sendmail is enabled, allo...	10.0	Critical
1	CVE-1999-0082	CWD ~root command in ftpd allows root access.	10.0	Critical
2	CVE-1999-1471	Buffer overflow in passwd in BSD based operati...	7.2	High
3	CVE-1999-1122	Vulnerability in restore in SunOS 4.0.3 and ea...	4.6	Medium
4	CVE-1999-1467	Vulnerability in rcp on SunOS 4.0.x allows rem...	10.0	Critical
5	CVE-1999-1506	Vulnerability in SMI Sendmail 4.0 and earlier,...	7.5	High
6	CVE-1999-0084	Certain NFS servers allow users to use mknod t...	8.4	High
7	CVE-2000-0388	Buffer overflow in FreeBSD libmytinfo library ...	7.5	High
8	CVE-1999-0209	The SunView (SunTools) selection_svc facility ...	5.0	Medium
9	CVE-1999-1198	BuildDisk program on NeXT systems before 2.0 d...	7.2	High

Fig 3. Dataset Description

The dataset that we collected provides a very standardized and reliable source for training our proposed machine learning model for vulnerability risk prediction [17].

C. Data Preprocessing

In this section we are describing the preprocessing steps that we applied to the raw CVE records that we collected from the NVD API, and all this is done before the model training and feature extraction. So the main goal of preprocessing is to transform the heterogeneous and unstructured vulnerability descriptions into a clean, consistent and machine readable format that is suitable for text based risk prediction.

The preprocessing workflow includes several stages. So the first thing is handling missing values and removing duplicate records, and these two steps are very important for making sure the data is clean and reliable. Then second thing is text normalization and label preparation, and this helps the model to work better and more accurately. And then the third thing is contextual outlier filtering and preparation for TF-IDF feature extraction, and the system does all these things automatically.

So all these steps together they improve the data quality, reduce the noise in the data, and also enhance the overall predictive performance of the machine learning pipeline. And we also found that similar text preprocessing strategies they are widely used in vulnerability prediction and NLP based security analysis studies [18][19].

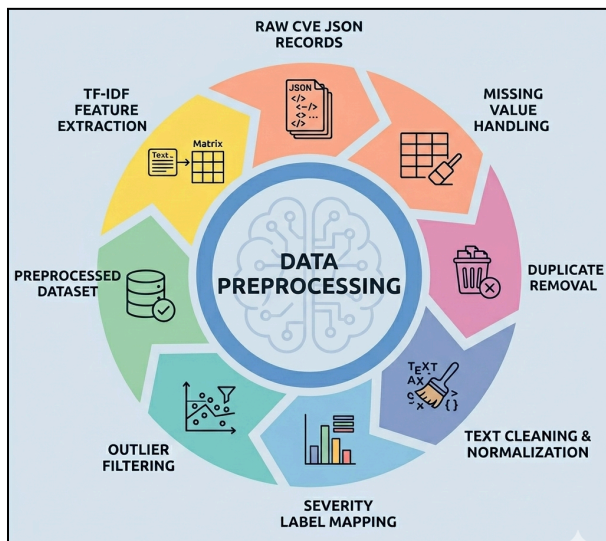


Fig. 4. Proposed Data Preprocessing Workflow

The overall preprocessing pipeline followed in the proposed framework is illustrated in **Fig. 4**.

C.1 Missing Value Handling

When we collected the vulnerability dataset from the NVD API, we noticed that a small number of entries had missing values. Most of these missing values showed up in the CVSS score field. In our preprocessing step, we found that out of the 2,000 records were collected, 34 entries had no CVSS score filled in. However, the good news was that all the other important fields we relied on — such as the CVE

ID, the vulnerability description, and the risk level — were complete and had no missing values at all.

To preserve dataset consistency and avoid unnecessary record removal, the missing numerical values in the `cvss_score` column were handled using mean imputation, as shown below:

```
df['cvss_score'] = df['cvss_score'].fillna(df['cvss_score'].mean())
```

This approach makes sure that the statistical characteristics of the dataset stay stable and also prevents any information loss during model training. And mean imputation is also very widely used for handling missing numerical attributes in machine learning pipelines[20].

And also on top of this, all the textual fields they were explicitly converted to string format, and we did this to make sure that everything stays uniform during the preprocessing and feature extraction steps.[21]

df.isnull().sum()		df.isnull().sum()	
cve_id	0	cve_id	0
description	0	description	0
cvss_score	34	cvss_score	0
risk_level	0	risk_level	0
text_len	0	text_len	0
description_clean	0	description_clean	0
dtype: int64		dtype: int64	

Fig.5. Before & After Missing Value Handling

C.2 Duplicate Record Verification

Since the dataset is collected using paginated API calls, there was a chance that duplicate CVE records could occur because of overlapping API boundaries or repeated updates. So to remove this kind of redundancy, what we did is we identified the duplicate entries using the unique CVE identifier which is the `cve_id` as the primary key[22].

When we actually analyzed the dataset, we found that there were no significant duplicate records in it. So because of this we kept all the entries as they are to preserve the completeness and integrity of the dataset.[23][24].

This step makes sure that the model is not getting affected by redundant patterns, class imbalance inflation or repeated vulnerability patterns during the model learning process[25].

```
df.duplicated(subset=['description']).sum()
np.int64(0)
```

Fig. 6. Duplicate Record Verification

C.3 Text Cleaning and Normalization

The textual vulnerability descriptions go through a normalization pipeline, and we did this to make sure that everything is consistent for NLP based feature extraction. So the preprocessing workflow involves several important stages. So first thing is that all the textual descriptions are converted to lowercase. Then the second thing is that URLs, hyperlinks, special characters and punctuation are all systematically removed from the text. After that whitespace normalization is also done to make sure the tokens are consistent. And throughout all this process, the alphanumeric tokens are carefully preserved so that the technical context of the vulnerability descriptions does not get lost. And then common stopwords are also filtered out so that only the most informative and security relevant terms are kept for feature extraction.[26][27][28].

URL and hyperlink removal was considered, but it was not required because the dataset did not contain significant hyperlink patterns.[29][30].

	cve_id	description	cvss_score	risk_level	risk_level_encoded
0	CVE-1999-0095	debug command sendmail enabled allowing attack...	10.0	Critical	0
1	CVE-1999-0082	cwd root command ftpd allows root access	10.0	Critical	0
2	CVE-1999-1471	buffer overflow passwd bsd based operating sys...	7.2	High	1
3	CVE-1999-1122	vulnerability restore sunos 403 earlier allows...	4.6	Medium	3
4	CVE-1999-1467	vulnerability rcp sunos 40x allows remote atta...	10.0	Critical	0
5	CVE-1999-1506	vulnerability smi sendmail 40 earlier sunos 40...	7.5	High	1
6	CVE-1999-0084	certain nfs servers allow users use mkndod gain...	8.4	High	1
7	CVE-2000-0388	buffer overflow freebsd libmytinfo library all...	7.5	High	1
8	CVE-1999-0209	sunview suntools selection_svc facility allows...	5.0	Medium	3
9	CVE-1999-1198	builddisk program systems 20 does prompt users...	7.2	High	1

Fig. 7. After Text Cleaning and Normalization

This normalization process reduces the linguistic noise in the data and also improves the effectiveness of TF-IDF vectorization[31][32].

C.4 Severity Label Preparation

The CVSS-based severity information obtained from the dataset is mapped into discrete categorical classes, which are Low, Medium, High and Critical. These severity labels are then used as the target classes for the multi-class classification model used in this study, and this helps the model in predicting the overall vulnerability risk level more effectively based on the input features.[33].

C.5 Contextual Outlier Filtering

As we can see in **Fig 8**, we performed boxplot analysis on both the CVSS scores and the description length, and we did this to find any potential outliers in the data[34].

For the CVSS score distribution, we found a few low end outliers that were near the range of 0.0 to 1.2. But what we decided is that we intentionally kept these values in the dataset and did not remove them. And the reason for this is that we wanted to preserve the complete range of vulnerability severity levels, so that the model can learn from both the low risk and the high risk situations properly.

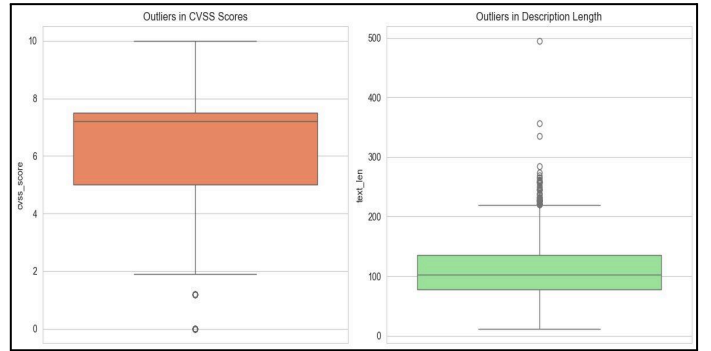


Fig. 8. Boxplot Analysis for CVSS Score and Description Length Outliers

Similarly, the description length boxplot showed several high end outliers that were going beyond 250 to 500 characters. But again we decided not to remove these records, and the reason for this is that longer descriptions often contain very critical and important technical details like exploit paths, affected functions and attack context. So removing them would have caused us to lose a lot of useful information[35].

Because of all this, we decided that no outlier removal was applied in this work at all. Keeping all the records helped in three main ways. First it preserved the diversity of the dataset. Second it maintained the contextual richness of the vulnerability descriptions. And third it maximized the data availability for TF-IDF feature extraction and XGBoost based risk prediction. So overall this was the right decision for making the model more accurate and reliable[36][37].

D. Feature Extraction

In this section we are describing the feature extraction process that we used to convert the preprocessed vulnerability descriptions into a numerical format that is suitable for model training[38].

The description field from the NVD dataset, we selected as the primary input feature which is X, and the reason for

this is that it contains the most important technical and semantic details of each vulnerability. And features like cve_id and cvss_score they were excluded from the input, and we did this to make sure that the model it predicts the risk based on the vulnerability semantics rather than just relying on the predefined scores.[39][40]

And then the target variable that we used for classification is risk_level_encoded, which basically represents the encoded severity labels and this is our y variable.

D.1 Feature Selection Basis

The feature selection process was based on the predictive relevance of each attribute in the dataset. So after looking at all the features, we finalized the following two. First is the description, which we selected as the primary predictive textual feature. And second is the risk_level_encoded, which we selected as the target classification label. So this selection strategy helps the model identify risk levels based primarily on the vulnerability semantics and also improves its capability to identify risk levels based purely on the vulnerability semantics[41].

D.2 Feature Extraction Technique

The following techniques were applied during the feature extraction process. First is TF-IDF Vectorization, which basically converts the cleaned descriptions into numerical vectors based on how important each term is. Second is Token Importance Weighting, which assigns higher weights to rare and security relevant terms so that they get more attention from the model. Third is Label Encoding, which converts the categorical severity labels into numerical classes so that XGBoost can process them properly. And fourth is Vocabulary Management, which constructs a feature space using the most informative tokens from the descriptions.

All these steps together they improve the models ability to capture the textual vulnerability patterns that are needed for accurate risk prediction

	Input Feature (X)	# Target Label (y)
0	debug command sendmail enabled	0
1	cwd root command ftpd allows root	0
2	buffer overflow passwd bsd based o	1
3	vulnerability restore sunos 403 earlie	3
4	vulnerability rcp sunos 40x allows rei	0

Fig. 9. Featured Dataset Representation

We can see in Fig 9, the dataset after feature selection consists of the input feature which is X and the corresponding target label which is y. And the finalized feature set mainly focuses on the semantic content of the

vulnerability descriptions. So by using TF-IDF based textual representation, the model learns the important technical terms and exploits related patterns directly from the vulnerability narrative. And this improves the systems ability to generalize to previously unseen vulnerabilities and also supports the zero-day risk prediction objective of our proposed CyberGuard framework.

E. Machine Learning Models

In this section we are evaluated on the supervised learning models that we implemented for vulnerability risk classification. So what we did is we tested multiple classifiers on the TF-IDF feature set to find out which model is the most suitable for multi-class risk prediction.

So the following candidate models were evaluated. First is Logistic Regression, which is a baseline linear classifier for sparse TF-IDF vectors. Second is Multinomial Naive Bayes, which is a probabilistic text classification model. Third is Random Forest, which is an ensemble tree based classifier. And fourth is XGBoost, which is an optimized gradient boosting framework.

Table 3.1 Comparative Performance Analysis

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.602	0.625	0.602	0.607
Random Forest	0.606	0.591	0.606	0.586
Naive Bayes	0.576	0.664	0.576	0.509
XGBoost	0.596	0.584	0.596	0.572

E.3 Final Model Selection

Even though Logistic Regression showed a slightly higher F1-score, we still selected XGBoost as the final model for the CyberGuard framework. And the main reason for this is that XGBoost has the ability to capture complex and non-linear exploit patterns, which is very critical for cybersecurity and zero-day vulnerability detection. And compared to linear models, XGBoost it also gives better generalization for evolving threat signatures and unseen attack behaviors.

And also its boosting mechanism improves the robustness of the model by iteratively reducing classification errors, which makes it much more suitable for real time risk prediction.

F. Model Training and Evaluation

The preprocessed dataset was divided into training and testing sets in a 75:25 ratio, and we did this to make sure the evaluation of our proposed framework is unbiased. And to preserve the proportional distribution of all the vulnerability risk classes which are Low, Medium, High and Critical, we applied stratified sampling during the split. To validate the robustness and stability of our proposed XGBoost model, we performed 5-fold stratified cross-validation on the training data. So what happens in this way is that the dataset gets divided into five equal parts and then the model is trained on four parts and validated on the remaining one part. And this process keeps repeating so that every part gets a chance to be the validation part. And this helped us make sure that the model performance stays consistent across different subsets of the dataset.

For the model training, we followed an integrated pipeline where the TF-IDF vectorization output was directly passed to the XGBoost classifier. And the model learned the vulnerability specific textual patterns through an iterative gradient boosting mechanism, where multiple decision trees were sequentially built one after another to minimize classification errors and improve the overall predictive performance.

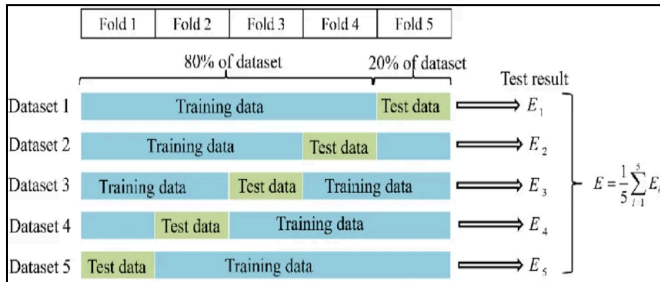


Fig. 10. Featured Dataset Representation

The final model was evaluated on the unseen test dataset using some standard classification metrics, which includes Accuracy, Precision, Recall and F1-Score. And on top of this, we also used a confusion matrix based error analysis to examine how the model is performing at the class level and also to find out if there is any misclassification happening between the closely related severity levels.

G. Rule-Based Scanner

The Rule-Based Scanner acts as the first line of defense. What this module essentially does is look directly at the source code and check it against a set of predefined rules to catch known insecure coding patterns and high-risk functions. We kept this layer separate from the machine learning part on purpose — while our ML layer predicts severity based on the meaning of the text, this scanner works in a more straightforward and deterministic way,

meaning it always gives the same result for the same code pattern.

Through this scanner, we are able to catch some of the most common and dangerous security flaws. These include things like SQL injection patterns, hardcoded credentials left inside the code, unsafe deserialization, and suspicious function calls that should not be there. We paid special attention to certain built-in functions that we know from experience are frequently misused — functions like `eval()`, `exec()`, `os.system()`, and `subprocess` calls where `shell=True` is enabled. These are particularly risky because attackers often exploit them to run arbitrary commands or gain remote access to a system.

We have summarized all the vulnerability signatures that our scanner monitors in Table 4.1.

Table 4.1 Vulnerability Signatures Used

Vulnerability Type	Target Function / Pattern	Risk Implication
Command Injection	<code>os.system()</code> , <code>subprocess.run()</code>	unauthorized shell execution
Code Injection	<code>eval()</code> , <code>exec()</code>	arbitrary code execution
Insecure Input	<code>input()</code> , unsafe file write	raw input exploitation
Unsafe Loading	<code>pickle.loads()</code> , <code>yaml.load()</code>	unsafe object deserialization

The output of this scanner is further combined with the ML prediction layer to improve the final risk scoring. So for example if the XGBoost model predicts a High risk level and at the same time the static scanner it also detects the presence of `eval()`, then the final risk gets elevated to Critical. This hybrid fusion improves the system's ability and really improves the systems ability to detect both known vulnerabilities and zero-day exploit patterns in a much better and more reliable way.

H. Intelligent Recommendation Layer

The Intelligent Recommendation System (IRS) provides developer-oriented remediation support by converting the raw risk predictions into clear and actionable security recommendations. This layer integrates the Gemini API-based generative AI module so that the system can generate easy to understand explanations and fix suggestions for the developers.

The model receives the predicted risk level, identified vulnerability signatures, and the relevant code snippets

from the previous layers. Based on this fused input, it generates a structured response that includes the risk explanation, possible business impact, secure coding recommendations, and zero-day threat insights, which makes the final output much more practical and useful during real development workflows.

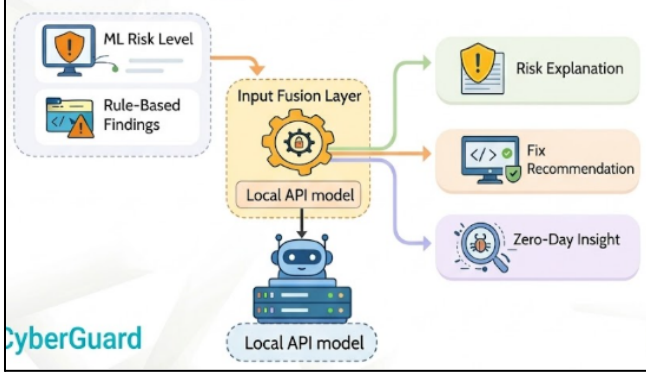


Fig. 11. Working of Intelligent Recommendation System

The recommendation workflow follows three stages:

First, the outputs of the ML prediction layer and rule-based scanner are fused; second, this information is passed to the Gemini API-based generative AI module using a security-focused prompt template; finally, the system produces human-readable fix suggestions and contextual threat explanations.

This intelligent layer significantly improves explainability, developer usability, and real-time remediation support.

The novelty of the proposed CyberGuard framework lies in its tri-hybrid integration of XGBoost-based severity prediction, rule-based static vulnerability detection, and Gemini API-driven zero-day remediation intelligence within a deployment-ready GitHub-integrated environment.

IV. RESULTS AND ANALYSIS

This section presents the experimental results and analytical findings of the proposed CyberGuard framework. The objective of this section is to evaluate the effectiveness of the system in terms of data characteristics, model performance, and vulnerability risk prediction capability.

This section includes the following key parts. The first one is Data Visualization and Exploratory Data Analysis. The second one is Performance Metrics. And third one is Result Discussion and Model Interpretation.

These analyses help validate the reliability of the proposed tri-hybrid framework for real-time zero-day vulnerability risk assessment and remediation support.

A. Data Visualization

Before model training, exploratory data analysis was performed to understand the distribution and characteristics of the collected vulnerability dataset.

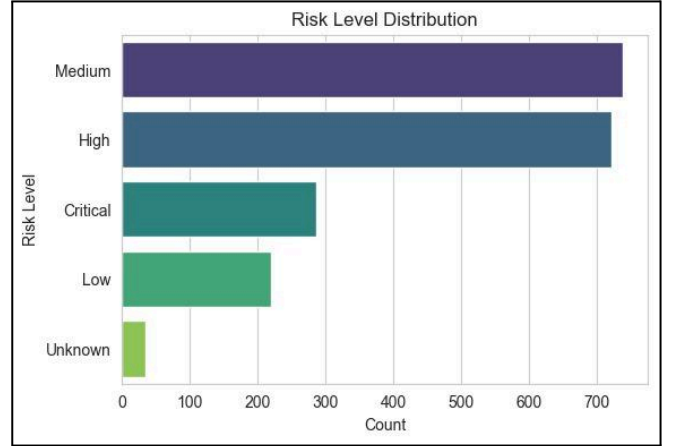


Fig. 12. Risk-level Distribution Chart

As shown in **Fig. 12**, the risk-level distribution shows that the dataset contains all major severity classes, with medium and High vulnerabilities appearing more frequently than Low and Critical categories. This conclusion helps identify biases and explains the use of stratified sampling during model training.

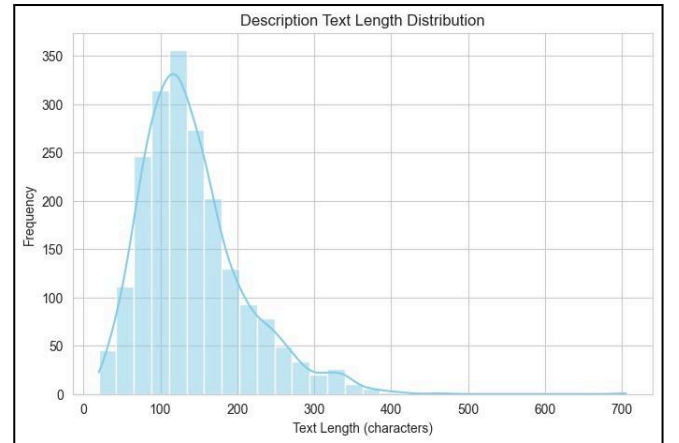


Fig. 13. CVSS Score Histogram

Further, **Fig. 13** illustrates the distribution of description text lengths. It can be observed that most vulnerability descriptions lie within the range of 80–200 characters, indicating that the textual records contain sufficient technical context for effective TF-IDF feature extraction.

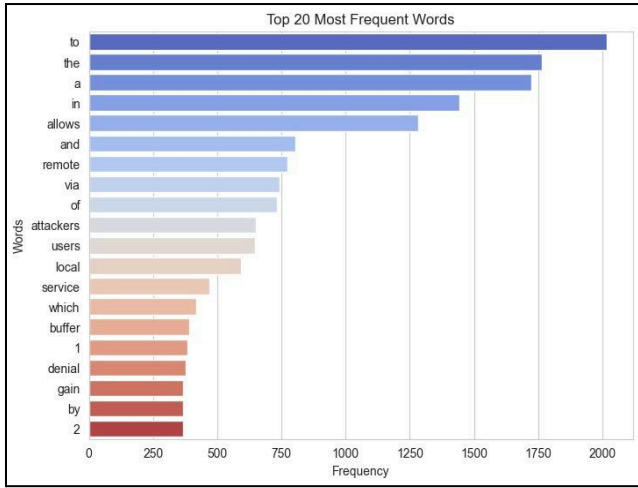


Fig. 14. Top Keywords Word Bar Plot

We also looked at the most frequently occurring words across our dataset, which we have presented in **Figure 14**. Words like "remote", "buffer", "attackers", and "denial" stood out clearly, and this told us something important — the data we collected is rich in security-relevant language. This gave us the confidence that our dataset carries exactly the kind of vocabulary that we needed for our model to learn from and make accurate vulnerability risk predictions.

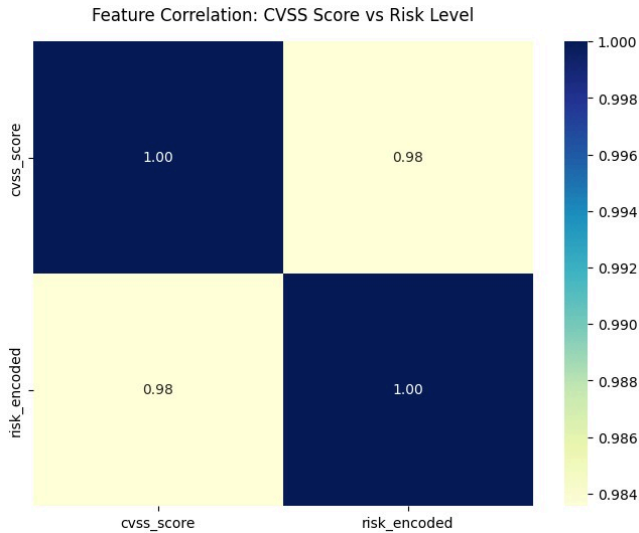


Fig. 15. Correlation Heatmap

We also examined the correlation between the CVSS score and the encoded risk level, which we have shown through a heatmap in **Figure 15**. What we found was a strong positive correlation of approximately 0.98 between the two, which essentially tells us that the numerical severity scores line up very closely with the categorical risk labels we assigned. Looking at all the exploratory data analysis results together, we can confidently say that the dataset we used is of good quality, covers a healthy diversity of vulnerability types, and is well suited for the CyberGuard framework we proposed.

B. Performance Metrics

To evaluate the effectiveness of the proposed CyberGuard system, multiple performance metrics were used like Accuracy, Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix analysis.

Among the evaluated models, Optimized XGBoost was selected as the final prediction engine due to its strong capability to capture non-linear relationships in high-dimensional TF-IDF feature space. In addition, its built-in regularization and parallel boosting architecture make it highly suitable for real-time cybersecurity applications.

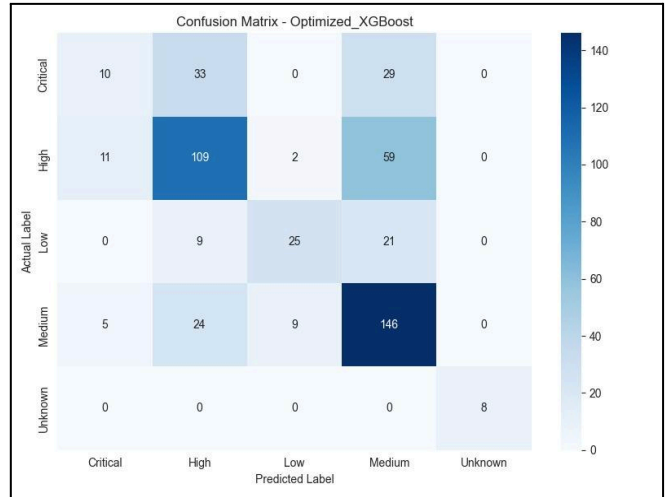


Fig. 16. Confusion Matrix for Optimized XGBoost

As shown in **Fig. 16**, the confusion matrix demonstrates that the model performs strongly in predicting the Medium and High risk classes, while maintaining reasonable classification performance across Critical and Low categories. The presence of some misclassifications between adjacent severity levels is expected due to semantic similarity in vulnerability descriptions.

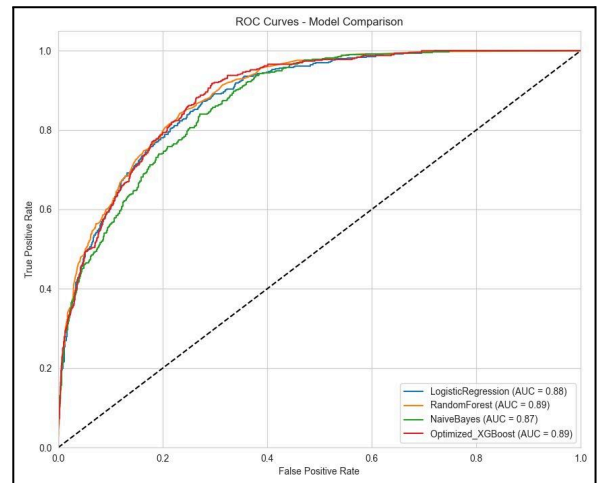


Fig. 17. ROC Curve Comparison of Candidate Models

Further, **Fig. 17** presents the ROC curve comparison across multiple candidate models. It can be seen that the optimized XGBoost achieved an AUC score of 0.89, which is among the highest values, confirming its strong biased capability for vulnerability risk prediction.

Now, from a cybersecurity standpoint, Recall and Precision matter more than overall accuracy. The reason for this is straightforward is if a real threat gets missed, meaning if a false negative occurs the consequences can be very serious. So when we selected our final model, we made sure it is accurate enough to catch as many threats as possible, while still keeping the alerts and not shattering developers with false alarms.

Table 4.1 Performance Metrics

Metric	Purpose	Security Significance
Accuracy	Overall correct predictions	System reliability
Precision	Correct high-risk alerts	Reduces false alarms
Recall	Detects real threats	Minimizes missed risks
F1-score	Balance of precision and recall	Stable security performance
ROC-AUC	Class separability	Model robustness

C. Result Discussion

The practical deployment results of CyberGuard demonstrate that the proposed framework is not limited to theoretical risk prediction but is also designed for real-world developer usability.

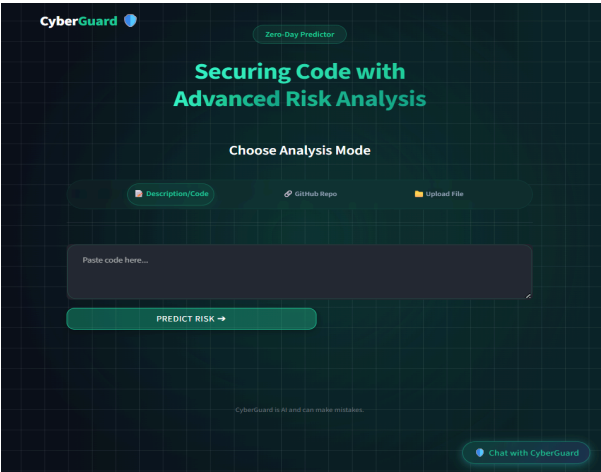


Fig. 18. CyberGuard Landing Page

This page is the main CyberGuard analysis interface where the user can choose the input mode like code, GitHub repo, or file upload and enter the required details. After clicking Predict Risk, the system analyzes the input and displays the vulnerability risk level with security insights.

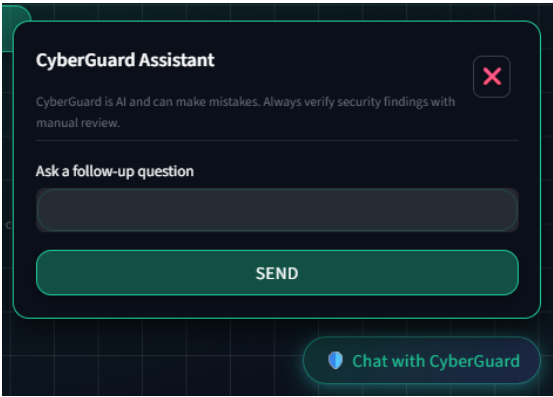


Fig. 19. CyberGuard Chatbot

This is the CyberGuard Assistant chatbot section where the user can ask follow-up questions related to the detected vulnerabilities and security risks. It helps in giving simple explanations, fix suggestions, and additional insights so that the user can better understand the issue in their code.

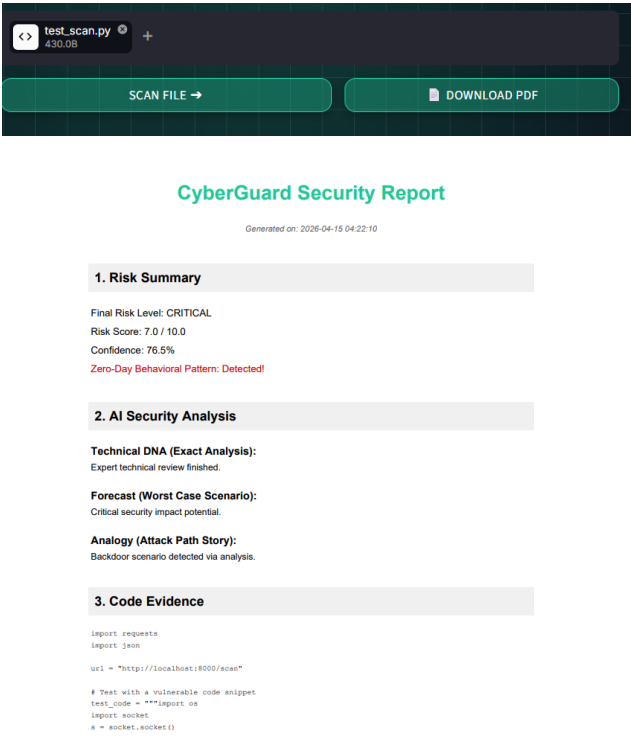


Fig. 20. Download Pdf and Evidence of Report

This section is used for file-based vulnerability analysis, where the user can upload a code file for scanning. After the scan is completed, the Download PDF option allows the user to download a detailed security report containing the detected vulnerabilities, risk level, and suggested fixes.

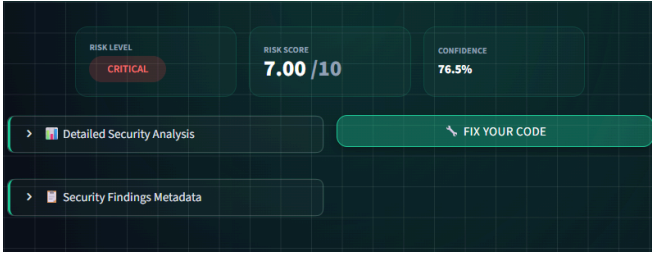


Fig. 21. CyberGuard Analysis

Further, as illustrated in **Fig. 21**, the predictive dashboard generates an instant output report containing the predicted risk level, severity class, and confidence-based threat insights, enabling quick decision-making during development and testing.

In addition, the framework gives three intelligent analysis perspectives for better understanding of threats. It includes Exact Analysis for technical explanation, Worst-Case Analysis for showing possible zero-day impact, and Story-Based Analysis for explaining the attack flow in simple language, which helps both technical and non-technical users to understand the risk better.

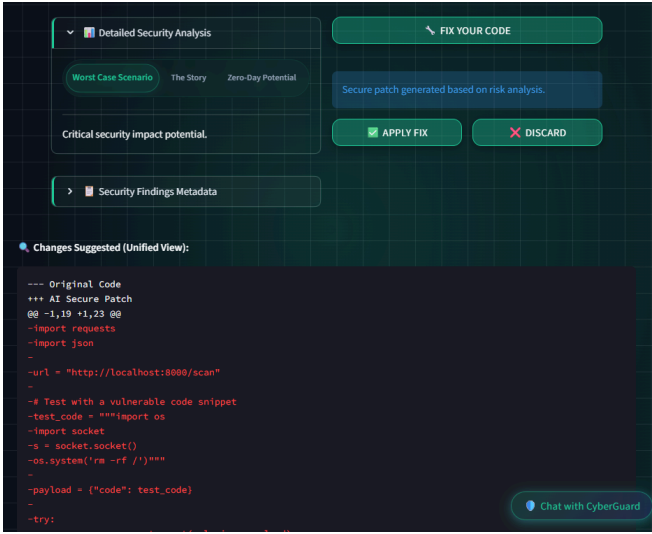


Fig. 22. Code level Auto Fix

Moreover, **Fig. 22** explains the intelligent recommendation interface, where the integrated API model generates real-time remediation steps, secure coding suggestions, and zero-day insights through chatbot-based interaction.

Overall, the result discussion confirms that CyberGuard performs effectively not only as a predictive model but also as a deployment-ready, developer-oriented cybersecurity solution.

V. DEPLOYMENT FRAMEWORK

When we built CyberGuard, we always had one goal in mind — it should not just be a research project that sits in a paper. We wanted it to work in the real world. It is a deployment-ready web application and Chrome extension-based security tool that developers can use without any complicated setup[42].

As we approached the deployment by keeping three things as our priority — it had to be accessible, it had to scale well, and it had to be easy enough for any developer to use without friction. We wanted CyberGuard to fit naturally into the way developers already work, whether that is through a web browser or directly through their code repositories.

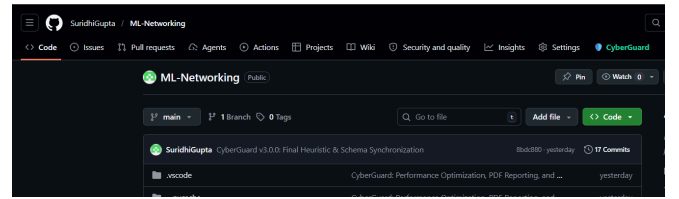


Fig. 23. Git-Hub Integrated Extension

For the actual deployment, we chose to build the frontend using Streamlit. The reason we picked Streamlit is simple, it is lightweight, interactive, and gets the job done without overcomplicating things. Through this interface, a developer can paste their code, scan a GitHub repository, or generate a detailed vulnerability report in real time. Streamlit is widely used for deploying machine learning applications as web services, which made it a natural and reliable choice for a deployment-ready cybersecurity tool like CyberGuard[43].

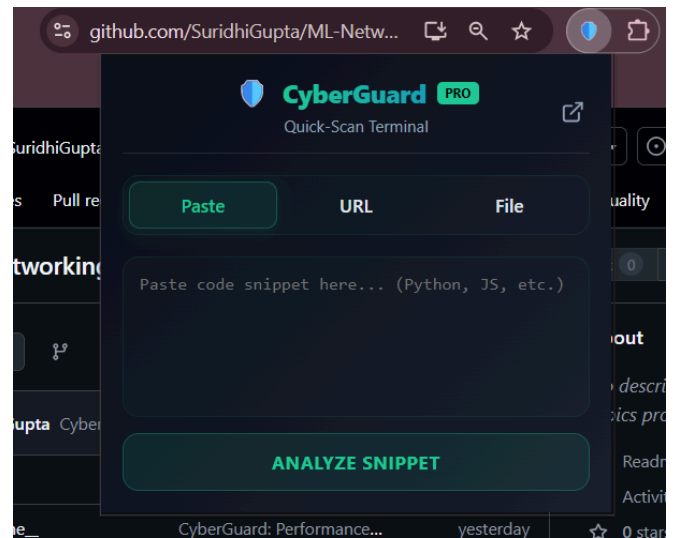


Fig. 24. CyberGuard Chrome Extension

The deployed application supports several core functionalities that are important for practical vulnerability assessment. It allows direct code input as well as

vulnerability description input, and along with that it also supports GitHub repository URL scanning for real-time analysis. The system also provides source file and ZIP upload analysis, XGBoost-based risk severity prediction, Gemini API-based recommendation and zero-day insight generation, and downloadable PDF security audit reporting, which together makes the whole CyberGuard framework much more useful and deployment ready for developers.

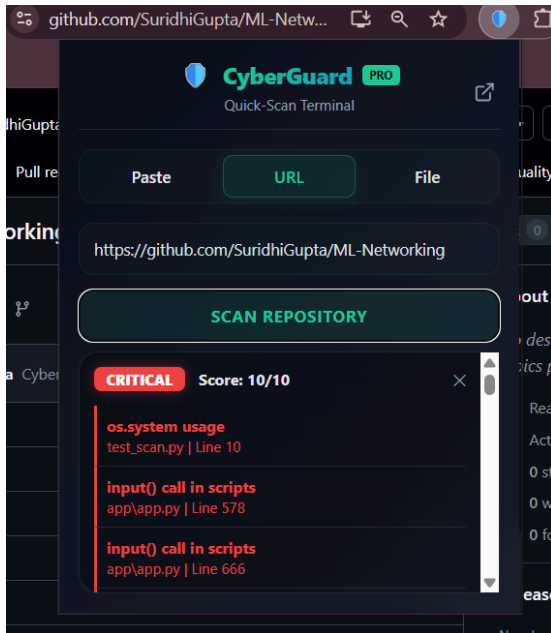


Fig. 25. Github URL Test

To support real-world software engineering workflows, the deployment system incorporates GitHub-integrated repository scanning, where developers can directly provide repository links for automated vulnerability assessment. This feature enables early stage risk analysis during development, testing, & code review time, that improves software security practices[44][45].

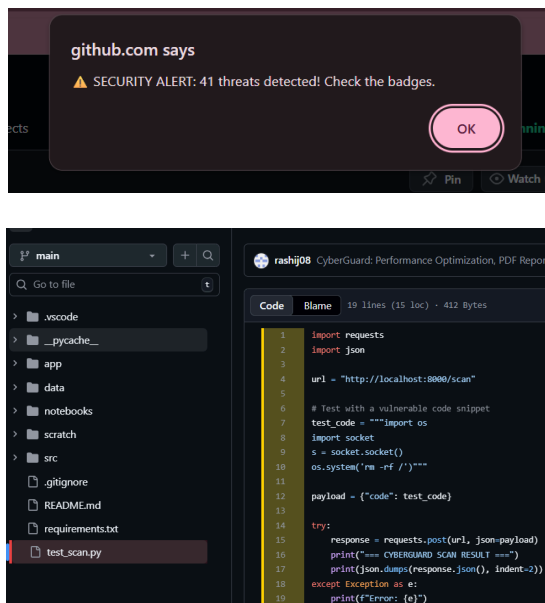
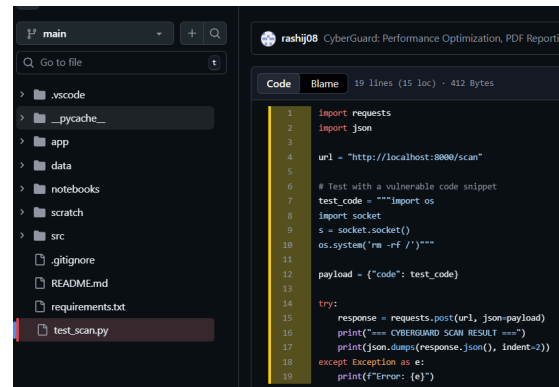


Fig. 26. Repo Files testing with Notification

In addition, the system also supports Chrome extension deployment, allowing users to initiate repository scans directly from GitHub pages through browser-level integration. This extension-based deployment model enhances practical usability and supports seamless integration with modern developer pipelines and repository review workflows [46].

The deployment architecture ensures that CyberGuard extends beyond a research prototype and functions as a practical, real-world security product for proactive zero-day vulnerability risk assessment and developer-oriented remediation support[47].



VI. CONCLUSION

In this paper, we presented CyberGuard — a framework we built to predict zero-day vulnerability risks in real time, so developers can stay ahead of security threats while writing software[48][49]. The core idea it design is a tri-hybrid architecture, which basically means we combined three things working together: XGBoost to predict how severe a vulnerability is, rule-based scanning to check the code, and a smart recommendation layer that tells you how to fix the issue and warns you about zero-day threats[50][51].

What we really wanted was for this to work in real developer workflows, not just in a lab. So we made sure CyberGuard supports pasting code directly, scanning GitHub repositories, and uploading files — whichever way a developer prefers to work. we also built it to be deployment-ready, meaning it can be used as a SaaS product, generates reports automatically, and and it is also deployed as a Chrome extension to make it even more accessible[52][53][54].

At the end of the day, what we set out to build — and believe we achieved — is a security tool that is scalable, proactive, and made with developers in mind. CyberGuard helps to catch vulnerabilities early & assess risk reliably which we hope contributes to building software that is stronger and harder to break[55].

VII. LIMITATIONS AND FUTURE WORK

This section highlights the planned enhancements that further strengthen the CyberGuard System & also highlights the practical limitations observed during current implementation phases[56].

A. Future Scope

The CyberGuard system provides a strong foundation for real time vulnerability prediction & zero day threat analysis. Some additional features were identified during development that could not be integrated due to time bound project constraints. These planned enhancements are there in Table 7.1.

Table 7.1 Future Scope and Planned Enhancements

Future Enhancement	Description
Historical Scan Data	Store previous scans for trend analysis, repeated vulnerability tracking, and comparative security review.
Audit Logs	Maintains timestamp-based logs of scans, user actions, predictions, and recommendations .
Scan Dashboard Analytics	Interactive dashboard showing severity trends, frequent threat types, and scan statistics over time.
RAG-Based Threat Knowledge Layer	Retrieval of real-time CVE, CWE, and exploit intelligence from external knowledge bases.
CI/CD Integration	We will integrate Automatic vulnerability checks during pull requests, code commits, and deployment pipelines.
Multi-Repository Monitoring	Enterprise-scale monitoring of multiple GitHub repositories under a unified dashboard.

As we have shown in Table 7.1 the future roadmap will be focusing on enhancing the system from a developer perspective security tool into an enterprise-grade deployment platform with long-term auditability, repository analytics, and automated remediation capabilities.

B. Practical Limitations

Even though our proposed system is showing strong performance and is ready for deployment, there are still some practical limitations that remain, and these are summarized in Table 7.2.

Table 7.2 Current Practical Limitations

Limitation	Description
Gemini API Response Time	Recommendation generation may take additional processing time depending on hardware and API response latency.
Hardware Dependency	Large-scale scans and API inference / recommendation performance depend on server resources such as CPU, RAM, and GPU support.
Dataset Generalization	The model performance may vary for highly domain-specific or newly emerging exploit descriptions.
Real-Time Scalability	High concurrent user load may require stronger backend server architecture for faster response.

As shown in Table 7.2, the current limitations are primarily practical rather than architectural. So for example the response time of the intelligent recommendation layer it depends a lot on the computational resources that are available and also the network latency.

So if we have access to high performance servers, dedicated GPUs and optimized deployment infrastructure, then the system it can give much faster response times and also support large scale real time vulnerability scanning in a much better way.

REFERENCES

- [1] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Computer Communications*, vol. 198, pp. 175–185, Jan. 2023.
- [2] Y. Guo, "A survey of machine learning-based zero-day attack detection," National Center for Biotechnology Information (PMC), 2023. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9890381/>
- [3] F. Abri, S. Siامي-Namini, M. A. Khanghah, F. M. Soltani, and A. S. Namin, "The performance of machine and deep learning classifiers in detecting zero-day vulnerabilities," *arXiv preprint arXiv:1911.09586*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.09586>

- [4] J. A. Harer et al., "Automated software vulnerability detection with machine learning," *arXiv preprint arXiv:1803.04497*, 2018. [Online]. Available: <https://arxiv.org/abs/1803.04497>
- [5] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, "From zero-shot machine learning to zero-day attack detection," *arXiv preprint arXiv:2109.14868*, 2021. [Online]. Available: <https://arxiv.org/abs/2109.14868>
- [6] M. Nkongolo and M. Tokmak, "Zero-day threats detection for critical infrastructures," *arXiv preprint arXiv:2306.06366*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.06366>
- [7] National Institute of Standards and Technology, "National Vulnerability Database (NVD)," [Online]. Available: <https://www.nist.gov/itl/nvd>
- [8] National Institute of Standards and Technology, "NVD CVE API v2.0," [Online]. Available: <https://services.nvd.nist.gov/rest/json/cves/2.0>
- [9] "Deploying machine learning models: A comparative study of Flask, FastAPI, and Streamlit," ResearchGate, 2025.
- [10] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," Google Research, 2016.
- [11] S. P. K. Reddy, U. Nagavelli, Y. S. Kiran, C. S. Kondoju, S. Bushmoni, and A. Yashaswi, "Deep learning for zero-day threat detection and mitigation," in *Proc. 2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)*, pp. 1362–1368, 2024.
- [12] N. Mohamed, H. Taherdoost, and M. Madanchian, "Comprehensive review of advanced machine learning techniques for detecting and mitigating zero-day exploits," *EAI Endorsed Transactions on Scalable Information Systems*, vol. 12, no. 1, p. 1, 2025.
- [13] M. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.08155>
- [14] A. Wakili and S. Bakkali, "ZeroDefense: An adaptive hybrid fusion-based intrusion detection system for zero-day threat detection in IoT networks," *Journal of Electronic Science and Technology*, p. 100345, 2026.
- [15] Y. Zhou et al., "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *arXiv preprint arXiv:1909.03496*, 2019. Available: <https://arxiv.org/abs/1909.03496>
- [16] A. Kutlimuratov et al., "A lightweight cascade-based framework for real-time zero-day attack detection," *Computers*, vol. 15, no. 3, p. 174, 2026.
- [17] Z. Li et al., "VulDeePecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018. Available: <https://arxiv.org/abs/1801.01681>
- [18] S. Akshaya, "ResNet50-based deep convolutional neural network for zero-day attack prediction and detection," *International Journal of Advanced Technology and Engineering Exploration*, vol. 12, no. 124, p. 507, 2025.
- [19] Z. Li et al., "SySeVR: A framework for using deep learning to detect software vulnerabilities," *arXiv preprint arXiv:1807.06756*, 2018. Available: <https://arxiv.org/abs/1807.06756>
- [20] A. Arun, A. S. Nair, and A. G. Sreedevi, "Zero day attack detection and simulation through deep learning techniques," in *Proc. 2024 14th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, pp. 852–857, 2024.
- [21] K. Kim et al., "Machine learning-based cyber threat detection using network traffic analysis," IEEE, 2020. Available: <https://ieeexplore.ieee.org/document/9090893>
- [22] A. Touré, Y. Imine, A. Semnont, T. Delot, and A. Gallais, "A framework for detecting zero-day exploits in network flows," *Computer Networks*, vol. 248, p. 110476, 2024.
- [23] F. Wang et al., "LlamaIDS: Real-time detection model of zero-day intrusions using large language models," in *Proc. 2025 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, pp. 768–769, 2025.
- [24] S. I. Berrios Vasquez, P. A. Hermosilla Monckton, D. I. Leiva Muñoz, and H. Allende, "Zero-day threat mitigation via deep learning in cloud environments," *Applied Sciences*, vol. 15, no. 14, p. 7885, 2025.
- [25] H. K. Hamid, "Hybrid CNN-LSTM model for real-time detection of zero-day attacks in heterogeneous IoT networks," *Journal of Information Hiding and Multimedia Signal Processing*, 2025.
- [26] G. Gowthami and S. S. Priscila, "Zero-day threat detection a machine learning paradigm for intrusion prevention," in *Proc. 2024 7th International Conference on Circuit Power and Computing Technologies (ICCPCT)*, vol. 1, pp. 852–857, 2024.
- [27] K. N. Karaca and A. Çetin, "Systematic review of current approaches and innovative solutions for combating zero-day vulnerabilities and zero-day attacks," *IEEE Access*, 2025.
- [28] A. De Paola, S. Drago, P. Ferraro, and G. Lo Re, "Detecting zero-day attacks under concept drift: An online unsupervised threat detection system," *CEUR Workshop Proceedings*, vol. 3731, 2024.
- [29] L. Y. Por et al., "A systematic literature review on AI-based methods and challenges in detecting zero-day attacks," *IEEE Access*, vol. 12, pp. 144150–144163, 2024.
- [30] W. Wang et al., "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," IEEE, 2017. Available: <https://ieeexplore.ieee.org/document/9152758>.
- [31] T. Y. Chiu, J. L. Chen, and C. Ahmadi, "Enhancing real-time malicious traffic detection in IoT networks via image-based convolutional neural networks: A scalable deep learning solution for zero-day threats," in *Proc. 2025 IEEE International Conference on Smart Internet of Things (SmartIoT)*, pp. 207–214, 2025.

- [32] M. Alkasassbeh, E. H. Omoush, M. Almseidin, and A. Aldweesh, "A self-adaptive intrusion detection system for zero-day attacks using deep Q-networks," *IEEE Access*, 2025.
- [33] S. Khandelwal and S. Shreejith, "Real-time zero-day intrusion detection system for automotive controller area network on FPGAs," in *Proc. 2023 IEEE 34th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*, pp. 139–146, 2023.
- [34] W. Wang et al., "An intrusion detection system based on autoencoder neural networks," IEEE, 2019.
Available: <https://ieeexplore.ieee.org/document/8736016>
- [35] Y. Kim et al., "Convolutional neural network-based intrusion detection system," IEEE, 2018.
Available: <https://ieeexplore.ieee.org/document/8436403>
- [36] A. S. Ashoor et al., "Intrusion detection system using support vector machine," IEEE, 2017.
Available: <https://ieeexplore.ieee.org/document/8012514>
- [37] K. Alam, M. F. Monir, M. J. Hossain, M. S. Uddin, and M. T. Habib, "Adaptive defense: Zero-day attack detection in NIDS with deep reinforcement learning," *IEEE Access*, 2025.
- [38] T. A. Mansur, "Zero-day guard: Self-supervised temporal correlation framework for zero-day threat detection in cloud platforms," in *Proc. 2025 IEEE First International Conference on Innovations in Engineering and Next-Generation Technologies for Sustainability (ICINVENTS)*, vol. 1, pp. 1–6, 2025.
- [39] B. M. Aslahi-Shahri et al., "A hybrid method for intrusion detection using random forest," IEEE, 2016.
Available: <https://ieeexplore.ieee.org/document/7470394>
- [40] A. B. Gupta, S. Akter, M. Islam, M. M. I. Javed, and J. Ferdous, "Smart defense: AI-powered adaptive IDs for real-time zero-day threat mitigation," IEEE, Piscataway, NJ, USA, Tech. Rep., 2023.
- [41] M. A. Alsuwaiket, "ZeroDay-LLM: A large language model framework for zero-day threat detection in cybersecurity," *Information*, vol. 16, no. 11, p. 939, 2025.
- [42] A. Al Siam et al., "Securing the unseen: A comprehensive exploration review of AI-powered models for zero-day attack detection," *Expert Systems*, vol. 43, no. 3, p. e70217, 2026.
- [43] O. Shukla, "Autonomous cyber defence in complex software ecosystems: A graph-based and AI-driven approach to zero-day threat mitigation," *Journal of Emerging Technologies and Innovation Management*, vol. 1, no. 1, pp. 01–10, 2025.
- [44] R. C. Sachan and R. K. Malviya, "Neural transformers for zero-day threat detection in real-time cybersecurity network traffic analysis," *IJGIS*, 2024.
- [45] S. I. Berrios Vasquez et al., "Zero-day threat mitigation via deep learning in cloud environments," *Applied Sciences*, 2025.
- [46] H. K. Hamid, "Hybrid CNN-LSTM model for real-time detection of zero-day attacks in heterogeneous IoT networks," *Journal of Information Hiding and Multimedia Signal Processing*, 2025.
- [47] K. N. Karaca and A. Çetin, "Systematic review of current approaches and innovative solutions for combating zero-day vulnerabilities," *IEEE Access*, 2025.
- [48] L. Y. Por et al., "A systematic literature review on AI-based methods and challenges in detecting zero-day attacks," *IEEE Access*, vol. 12, pp. 144150–144163, 2024.
- [49] K. Alam et al., "Adaptive defense: Zero-day attack detection in NIDS with deep reinforcement learning," *IEEE Access*, 2025.
- [50] T. A. Mansur, "Zero-day guard: Self-supervised temporal correlation framework for zero-day threat detection in cloud platforms," IEEE, 2025.